

서비스데스크 서버 & 웹위젯 개발 가이드 (주니어 개발자용)

본 문서는 **01.archetecture.md** 의 아키텍처를 기반으로, 서비스데스크 서버와 웹위젯을 처음 개발하는 주니어 개발자를 위한 실무 가이드입니다.

읽기 전 권장 순서: ① 이 문서 1~4장(아키텍처/책임 이해) → ② **01.archetecture.md**(원본 요구) → ③ 5장 이후(실무 팁/체크리스트).

0. 용어 사전

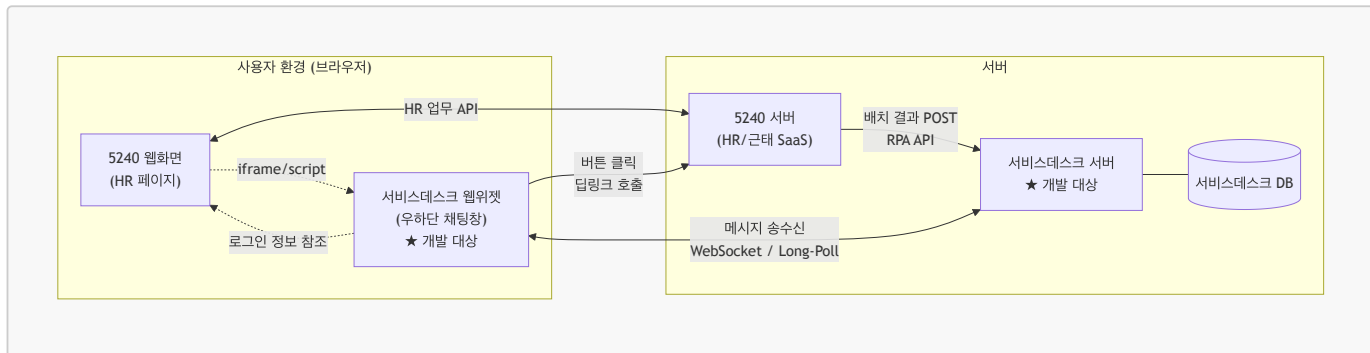
먼저 등장하는 시스템과 용어를 정리합니다. 문서 내내 같은 의미로 사용합니다.

용어	설명
5240 서버	HR/근태 SaaS 백엔드. 출퇴근/휴가 등 인사 업무 처리. 우리(서비스데스크)는 외부 시스템으로 취급.
5240 웹화면	사용자가 브라우저로 접속하는 5240 프론트엔드. 우리 웹위젯이 이 화면 안에 박힌다.
서비스데스크 서버	우리가 개발할 백엔드. RPA API 수신, 메시지 저장, 웹위젯으로 메시지 전송 담당.
서비스데스크 웹위젯	우리가 개발할 프론트엔드. 5240 웹화면 우하단에 떠 있는 채팅창 UI.
RPA API	5240 서버가 우리 서버로 이벤트(출근체크, 휴가신청 안내 등)를 POST 하는 인입 API.
chatbotId	챗봇 인스턴스 식별자. 회사/테넌트 구분에 사용.
loginId	5240 사용자 로그인 ID. 우리 시스템과 5240 양쪽에서 같은 사용자를 가리키는 키.
rpaCode	메시지 종류 코드(예: HR_PC_ON). 어떤 버튼/카드를 보여줄지 결정.
딥링크 URL	버튼 클릭 시 호출하는 5240 모바일 화면 URL(예: MobileLeaveAppl.do).

1. 전체 시스템 아키텍처

1.1 구성 요소

본 프로젝트는 **4개 시스템**이 협력합니다. 이 중 **우리가 개발하는 것은 2개**(서비스데스크 서버, 웹위젯)입니다.



중요 포인트

- 우리 웹위젯은 **5240 웹화면 안에 임베드**되므로, 같은 브라우저 컨텍스트에서 **5240의 로그인 정보(쿠키/세션)**에 접근할 수 있다고 가정합니다.
- 우리 서버는 5240 서버에게 직접 API를 호출하지 않습니다(단방향: 5240 → 우리 서버). 사용자 액션은 **웹위젯이 5240 딥링크 URL 을 iframe/팝업으로 직접 연다**(코드에서 fetch/XHR 로 호출하는 것이 아님).

1.2 시스템 간 통신 채널 요약

구간	방향	프로토콜	용도
5240 서버 → 서비스데스크 서버	단방향	HTTPS POST (RPA API)	배치 이벤트 통지
서비스데스크 서버 ↔ 웹위젯	양방향	WebSocket 또는 SSE/Long-Polling	메시지 푸시, 채팅 송신
웹위젯 → 5240 서버	단방향	HTTPS GET (페이지 로드)	버튼 클릭 → iframe/팝업으로 HR 화면 열기
5240 웹화면 ↔ 웹위젯	양방향	동일 origin 또는 postMessage	로그인 정보 전달

2. 서비스데스크 서버 — 기능 및 개발 방법

2.1 핵심 책임

1. 5240 서버가 호출할 RPA API 제공 (POST /chatbot/api/v1/rpa/{chatbotId}/messages)
2. 수신한 메시지를 DB에 영구 저장 (대화 이력 관리)
3. 해당 사용자의 웹위젯으로 메시지 전달 (실시간 푸시 또는 폴링)
4. 웹위젯과의 양방향 채팅 세션 관리 (연결, 인증, 메시지 라우팅)
5. 채팅 입력 시 추천단어(intent) 제공 (HR_PC_ON 등으로 매핑되는 키워드 사전)

2.2 RPA API 스펙

본 스펙은 5240 측 실제 호출 소스(2.티그리스 챗봇 api 호출 소스.txt, SendController#getChatbotQueueList, #getChatbotPushQueueList)에서 확정된 내용입니다.

```
POST {서비스데스크Domain}/chatbot/api/v1/rpa/{chatbotId}/messages
Content-Type: application/x-www-form-urlencoded
charset:      utf-8
```

[Request Body – form-urlencoded]

```
rpaCode       : 인터페이스 RPA 코드 (아래 표 참조, 빈 문자열 가능)
messageCode   : 메시지 코드 (예: TAA_100, 또는 결재의 경우 msgId)
message       : 전달할 메시지 본문
loginId       : 5240 사용자 로그인 ID (이메일 형태, 평문)
overtime      : 임시 연장 시간(분) – 퇴근 연장 시나리오에서만 의미
startTime     : 시업 일시 (yyyymmddhh24mi, PC-ON 기준)
endTime       : 종업 일시 (yyyymmddhh24mi, PC-OFF 기준)
userType      : 사용자 구분 – EXCEPTION | GENERAL
clientId      : 챗봇 API 클라이언트 ID (사전 발급)
secret        : 챗봇 API 보안 키 (사전 발급)
```

[Response 200 – 5240 측에서 파싱하는 필드]

```
{
  "code":      "0",           // 문자열로 파싱됨 (Java: getAsString())
  "message":   "OK"
}
```

인증 방식 (확정)

- 별도 헤더 인증 없음. **body** 파라미터 **clientId + secret** 으로 검증.
- 따라서 서비스데스크 서버는 (**clientId, secret**) 쌍을 발급/저장하고, 모든 RPA 요청에서 이 쌍이 유효한지 확인해야 합니다.
- 평문 전송이므로 반드시 **HTTPS** 강제. 추가로 IP allowlist 권장.

chatbotId

- URL path 변수. 5240 운영 사례에서는 **kori** 가 사용됨(.../rpa/kori/messages).
- 테넌트/사이트 단위 식별자로 서비스데스크가 발급·관리.

rpaCode 매핑표 (HR/근태 배치 — **getChatbotQueueList**)

5240 내부 코드	전송되는 rpaCode	의미
HR_RPA_090	`` (빈 문자열)	근무시간 안내 (알림만)
HR_RPA_100	HR_PC_ON	출근체크 + PC-ON
HR_RPA_110	`` (빈 문자열)	퇴근시간 안내 (알림만)
HR_RPA_120/130/140	HR_GET_OFF_WORK	퇴근 체크 / 연장 1·2차
HR_RPA_800	HR_PC_OFF	PC 종료

rpaCode 매핑표 (결재 상시 이벤트 — **getChatbotPushQueueList**)

5240 내부 코드	전송되는 rpaCode	의미
ELA_REQST_SIGN / EAP_REQST_SIGN	HR_APPROVAL_REQUEST	결재 요청
ELA_REQST_SANCTION / EAP_REQST_SANCTION	HR_APPROVAL_COMPLETED	결재 완료
ELA_RECALL / EAP_RECALL	HR_APPROVAL_WITHDRAW	결재 회수
ELA_REQST_REJECT / EAP_REQST_REJECT	HR_APPROVAL_RETURN	결재 반려
그 외	`` (빈 문자열)	기타 푸시

⚠️ 유의

- 5240 소스는 응답에서 최상위 **code / message** 두 필드만 읽습니다. **data[]** 같은 상세 응답을 내려도 5240 큐 처리 로직은 무시합니다. → 서비스데스크 서버 입장에서 응답 스키마는 { **code, message** } 만 합의되어 있다고 보면 됩니다.
- 5240 측이 **messageCode** 에 넣는 값은 두 메서드에서 다릅니다. 배치(**getChatbotQueueList**)는 **content** 의 첫 토큰(예: **TAA_100**), 결재(**getChatbotPushQueueList**)는 **msgId** 자체. 서비스데스크에서는 **messageCode** 를 식별자가 아닌 **메시지 종류 힌트**로만 취급하고, 역등성 키는 별도로(예: **seq** 또는 (**loginId, sortKey**)) 두는 것이 안전합니다.
- **loginId** 는 5240 측에서 이메일 형식으로 송신됩니다(**xxx@tigrison.com**). 우리 DB의 **login_id** 컬럼 타입/길이도 이를 수용해야 합니다.

2.3 권장 모듈 구성 (예시, 참조만)

```

service-desk-server/
├── api/
│   ├── rpa.controller.ts      # POST RPA API 진입점
│   └── widget.controller.ts   # 웹위젯 REST/WebSocket 진입점
├── service/
│   └── message.service.ts     # 메시지 저장/조회

```

```

├── intent.service.ts      # rpaCode → 버튼 카드 변환, 키워드 → intent 매핑
├── push.service.ts       # 웹위젯으로 푸시 (WebSocket broker)
├── repository/
│   ├── message.repository.ts
│   └── session.repository.ts
├── domain/
│   ├── Message.ts        # 메시지 도메인 모델
│   └── ButtonCard.ts     # 위젯에 보낼 버튼 카드 DTO
├── ws/
│   └── widget.gateway.ts  # WebSocket 연결 관리

```

2.4 데이터 모델 (예시, 참조만)

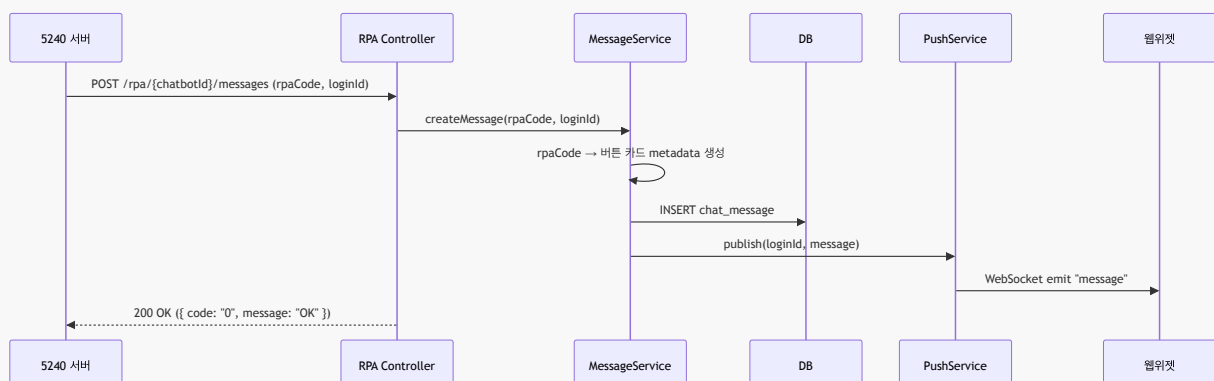
```

-- 채팅 메시지
CREATE TABLE chat_message (
  message_id      VARCHAR(64) PRIMARY KEY,
  chatbot_id      VARCHAR(64) NOT NULL,
  login_id        VARCHAR(64) NOT NULL,
  sender          ENUM('user', 'system', 'rpa') NOT NULL,
  rpa_code        VARCHAR(32),
  response_code   VARCHAR(32),
  content         TEXT,
  metadata        JSON,
  read_yn         CHAR(1) DEFAULT 'N',
  sort_key        BIGINT,
  create_dt       BIGINT NOT NULL,
  INDEX idx_user (chatbot_id, login_id, sort_key)
);

-- 웹위젯 연결 세션 (WebSocket 사용 시)
CREATE TABLE widget_session (
  session_id      VARCHAR(64) PRIMARY KEY,
  chatbot_id      VARCHAR(64) NOT NULL,
  login_id        VARCHAR(64) NOT NULL,
  connected_at    BIGINT NOT NULL,
  last_ping_at    BIGINT,
  node_id         VARCHAR(64) -- 멀티노드 환경에서 어떤 서버 인스턴스에 붙어있는지
);

```

2.5 RPA → 위젯 푸시 시퀀스 (서버 내부)



2.6 권장 기술 스택 (팀 선택지)

항목	권장 후보	이유
언어/런타임	Node.js (NestJS) / Spring Boot	기존 팀 스택과 일치하는 것 선택
DB	MySQL / PostgreSQL	트랜잭션·인덱스 안정성
실시간 채널	WebSocket (Socket.IO 또는 native)	양방향, 저지연. SSE는 단방향이라 채팅 송신에 별도 REST 필요
메시지 브로커	Redis Pub/Sub	멀티노드 환경에서 push 분기
캐시	Redis	세션·intent 사전 캐싱

3. 서비스데스크 웹위젯 — 기능 및 개발 방법

3.1 핵심 책임

1. **5240** 웹화면 우하단에 채팅 UI 렌더링 (플로팅 버튼 + 패널)
2. 서비스데스크 서버와 양방향 연결 유지 (WebSocket 재연결 포함)
3. 서버에서 받은 메시지를 채팅창에 표시
 - 일반 텍스트
 - **rpaCode** 기반 버튼 카드 (출근/퇴근/휴가신청 등)
4. 사용자 입력 시 추천단어(자동완성) 노출
5. 버튼 클릭 시 **5240** 딥링크 URL 호출 → 위젯 영역에 팝업/iframe으로 **5240** 화면 표시
6. **5240** 로그인 정보(loginId 등) 자동 획득 (사용자가 다시 로그인하지 않게)

3.2 권장 배포 형태 (예시, 참조만)

웹위젯은 **5240** 페이지에 한 줄 `<script>` 로 삽입되는 형태가 일반적입니다.

```
<!-- 5240 웹화면 어딘가에 -->
<script src="https://{서비스데스크Domain}/widget/loader.js"
  data-chatbot-id="acme-corp"></script>
```

`loader.js` 가 하는 일:

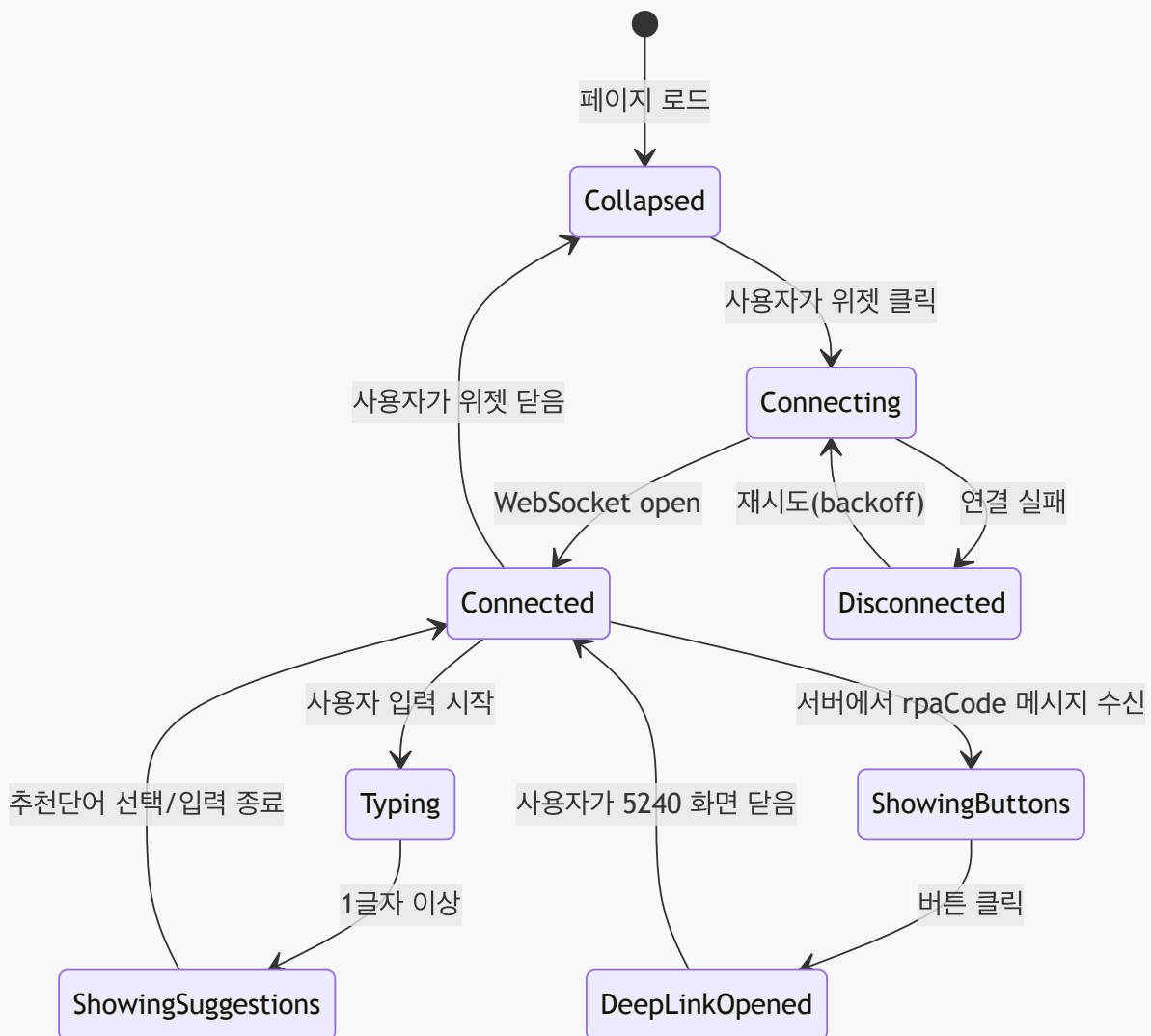
1. 호스트 페이지(5240)에 `<iframe>` 또는 Shadow DOM 컨테이너를 우하단에 주입
2. 내부에서 위젯 SPA(React/Vue/Svelte) 로딩
3. 호스트 페이지에서 `loginId` 추출 → 위젯으로 전달

3.3 권장 디렉터리 구조 (예: React + Vite) (예시, 참조만)

```
service-desk-widget/
├── public/
│   └── loader.js           # 호스트 페이지가 로드하는 진입 스크립트
├── src/
│   ├── main.tsx           # 위젯 앱 마운트
│   ├── App.tsx
│   ├── components/
│   │   ├── ChatPanel.tsx
│   │   ├── MessageList.tsx
│   │   ├── MessageBubble.tsx
│   │   └── ButtonCard.tsx  # rpaCode 기반 버튼 카드 렌더링
```

├─ Suggestions.tsx	# 추천단어 노출
├─ DeepLinkFrame.tsx	# 5240 화면을 띄울 iframe/팝업
├─ hooks/	
│ └─ useWebSocket.ts	
│ └─ useAuth.ts	# 5240 로그인 정보 획득
├─ services/	
│ └─ api.ts	# REST 호출 (이력 조회 등)
│ └─ ws.ts	# WebSocket 클라이언트
├─ intent/	
│ └─ suggestions.ts	# "출" 입력 → ["출근", "출장신청"] 추천
├─ types/	
│ └─ message.ts	
└─ index.html	

3.4 화면 상태 머신 (간소화)



3.5 rpaCode → UI 매핑 (예시)

서버가 보낸 메시지의 **rpaCode** 또는 **responseCode** 와 **metadata** 를 보고 어떤 버튼 카드를 그릴지 결정합니다.

위젯 버튼 → 5240 딥링크 URL 매핑 카탈로그 (canonical)

#	화면	경로	절대 URL 예시
1	출퇴근체크 (모바일메인)	/MobileMain.do	https://api.5240.cloud/MobileMain.do
2	휴가신청	/MobileLeaveAppl.do	https://api.5240.cloud/MobileLeaveAppl.do
3	연장근무신청	/MobileOvertimeAppl.do	https://api.5240.cloud/MobileOvertimeAppl.do
4	출장신청	/MobileBusinessAppl.do	https://api.5240.cloud/MobileBusinessAppl.do
5	조퇴/외출신청	/MobileHalfLeaveAppl.do	https://api.5240.cloud/MobileHalfLeaveAppl.do
6	출퇴근변경	/MobileWorkTimeChgAppl.do	https://api.5240.cloud/MobileWorkTimeChgAppl.do
7	경조휴가신청	/MobileConLeaveAppl.do	https://api.5240.cloud/MobileConLeaveAppl.do
8	결재함	/MobileApprovalBox.do	https://api.5240.cloud/MobileApprovalBox.do

#	화면	경로	절대 URL 예시
9	근무일정조회	/MobileDclzWorkSearchCldr.do	https://api.5240.cloud/MobileDclzWorkSearchCldr.do
10	경조신청	/MobileCtsmnAppl.do	https://api.5240.cloud/MobileCtsmnAppl.do

💡 URL 의 origin(<https://api.5240.cloud>)은 환경별로 달라질 수 있으므로 위젯 코드에 절대 URL을 하드코딩하지 말 것. origin 은 환경 변수(예: `VITE_HR_BASE_URL`)로 두고, 경로만 매핑 테이블로 관리하거나, 아예 서버 응답 `metadata.buttons[].url` 을 그대로 사용하는 것을 권장합니다.

예시 코드 (위 카탈로그를 기반으로 한 매핑)

```
// src/intent/buttonMap.ts
const HR_BASE = import.meta.env.VITE_HR_BASE_URL; // 예: https://api.5240.cloud

// rpaCode → 버튼 카드 (RPA 메시지 동반 버튼)
export const BUTTON_MAP = {
  HR_PC_ON: { label: '출근 체크', path: '/MobileMain.do' },
  HR_GET_OFF_WORK: { label: '퇴근 체크', path: '/MobileMain.do' },
  HR_PC_OFF: { label: 'PC 종료', path: '/MobileMain.do' },
  HR_APPROVAL_REQUEST: { label: '결재함', path: '/MobileApprovalBox.do' },
  HR_APPROVAL_COMPLETED: { label: '결재함', path: '/MobileApprovalBox.do' },
  HR_APPROVAL_WITHDRAW: { label: '결재함', path: '/MobileApprovalBox.do' },
  HR_APPROVAL_RETURN: { label: '결재함', path: '/MobileApprovalBox.do' },
} as const;

// 추천단어 → 버튼 카드
export const SUGGESTION_TO_BUTTONS = {
  '휴가': [{ label: '휴가신청', path: '/MobileLeaveAppl.do' }],
  '출장': [{ label: '출장신청', path: '/MobileBusinessAppl.do' }],
  '연장근무': [{ label: '연장근무신청', path: '/MobileOvertimeAppl.do' }],
  '조퇴': [{ label: '조퇴/외출신청', path: '/MobileHalfLeaveAppl.do' }],
  '외출': [{ label: '조퇴/외출신청', path: '/MobileHalfLeaveAppl.do' }],
  '출퇴근변경': [{ label: '출퇴근변경', path: '/MobileWorkTimeChgAppl.do' }],
  '경조': [
    { label: '경조휴가신청', path: '/MobileConLeaveAppl.do' },
    { label: '경조금신청', path: '/MobileCtsmnAppl.do' },
  ],
  '근무일정': [{ label: '근무일정조회', path: '/MobileDclzWorkSearchCldr.do' }],
  '결재': [{ label: '결재함', path: '/MobileApprovalBox.do' }],
} as const;

export const toAbsoluteUrl = (path: string) => `${HR_BASE}${path}`;
```


⚠ 위 `BUTTON_MAP` 의 `rpaCode` 값은 2.2 RPA API 스펙의 매핑표(`HR_PC_ON` / `HR_GET_OFF_WORK` / `HR_PC_OFF` / `HR_APPROVAL_*`)와 일치시켜야 합니다. 5240 측에서 전송되지 않는 코드(예: `HR_MENU`, `HR_GO_TO_WORK`)는 카탈로그에서 제외했습니다 — 추후 5240 합의 시 추가.

3.6 추천단어(자동완성) UX

1. 사용자가 채팅 입력란에 글자를 타이핑
2. **2글자 이상**부터 prefix 매칭으로 추천단어 목록 표시 (서버에서 받거나, 위젯 내장 사전)
3. 사용자가 추천단어 클릭 → 해당 단어에 매핑된 **버튼 카드를 채팅창에 추가**
4. 버튼 클릭 → 딥링크 호출

💡 추천단어 사전은 위젯에 **정적으로 내장**해도 되지만, 서버 API로 동적 조회하는 편이 운영 중 변경이 쉽습니다.

3.7 딥링크 화면 띄우기

요구사항: "위젯 화면 영역에 5240 화면이 팝업으로 뜬다"

구현 방법은 두 가지 — 5240 측 정책에 따라 선택:

방식	장점	단점
<code>window.open(url, '_blank', 'width=480,height=650')</code>	구현 간단, CSP/iframe 제약 없음	팝업 차단 가능, 위젯과 분리된 창
위젯 내부 <code><iframe src={url}></code>	위젯 안에서 보임(UX 통일)	5240 측 <code>X-Frame-Options/CSP frame-ancestors</code> 허용 필요

→ **iframe** 방식이 요구사항(위젯 영역에 팝업)에 더 부합합니다. 5240 운영팀과 `frame-ancestors` 헤더 허용을 사전 협의해야 합니다.

3.8 로그인 정보 획득

시나리오	방법
위젯이 5240과 동일 origin 으로 서빙	<code>document.cookie</code> / <code>window.parent.userInfo</code> 등 직접 접근
위젯이 별도 origin iframe	호스트 페이지가 <code>postMessage({ type: 'AUTH', loginId })</code> 로 전달

→ 위젯 초기화 시 `loginId` 가 없으면 채팅 기능을 비활성화하고 안내 메시지를 띄웁니다.

3.9 버튼 클릭 동작 흐름 (전체 정리)

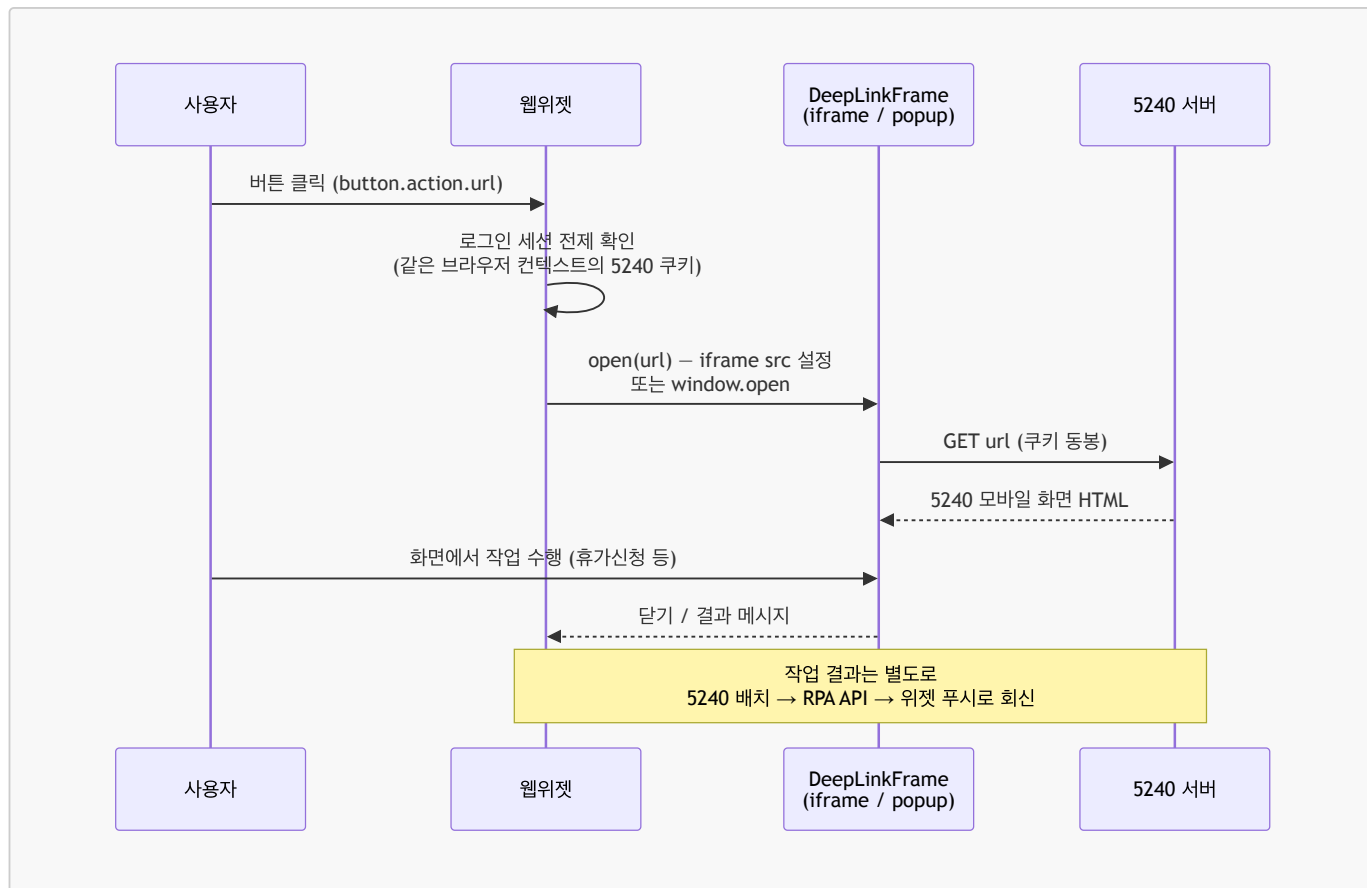
3.5(rpaCode → UI 매핑), 3.7(딥링크 띄우기), 3.8(로그인 정보)이 결합되는 **end-to-end 흐름**을 한 곳에 정리합니다.

버튼이 표시되는 두 경로

1. **RPA 메시지 동반 버튼**: 5240 → 서비스데스크 RPA API 푸시 → 위젯 카드 렌더링 (예: `HR_PC_ON` → "출근 체크")
2. **추천단어 선택 후 버튼**: 사용자가 채팅창에 "출" 입력 → 추천단어 노출 → 선택 시 매핑된 버튼 카드 추가

→ 두 경로 모두 결국 동일한 자료구조의 버튼으로 수렴합니다: `{ label, url, (선택) target }`.

클릭 시퀀스



누가 무엇을 담당하나

단계	담당	비고
<code>rpaCode</code> → 버튼 메타데이터(<code>label</code> , <code>url</code>) 변환	서비스데스크 서버 (<code>intent.service</code>)	서버가 응답 <code>metadata</code> 로 내려주는 게 권장
버튼 카드 렌더링	위젯 (<code>ButtonCard.tsx</code>)	—
URL 호출 / 화면 표시	위젯 (<code>DeepLinkFrame.tsx</code>)	iframe 우선, 차단 시 popup 폴백
인증/세션	5240 웹화면	위젯은 같은 origin 쿠키만 활용

놓치면 안 되는 설계 포인트

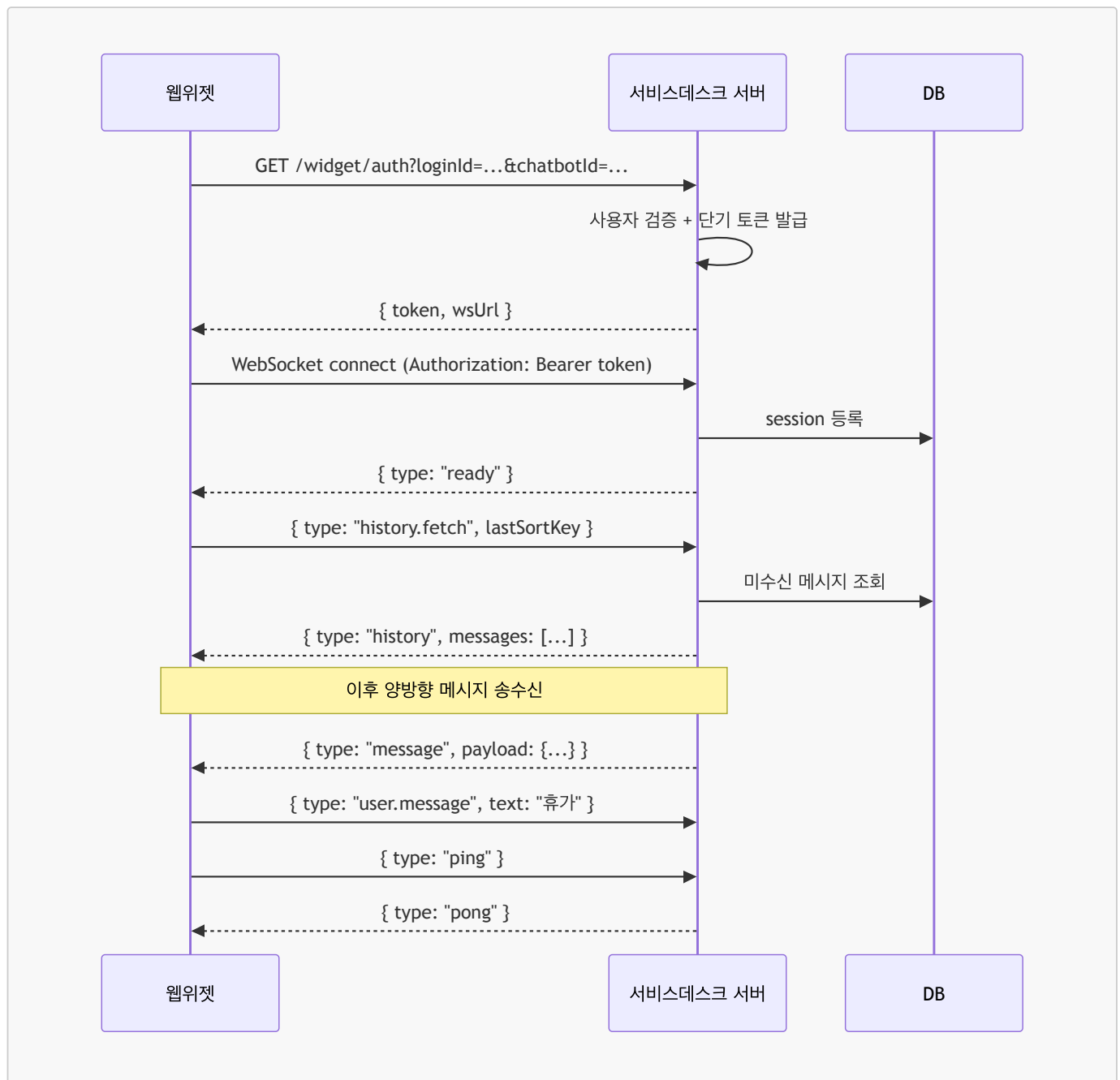
- 위젯은 5240 API 를 코드에서 직접 호출(`fetch/XHR`)하지 않는다. URL을 iframe/팝업으로 열어 브라우저가 페이지를 로드하게 할 뿐 — 요청 본문/인증 처리는 5240 화면(iframe 안)이 직접 수행합니다.
- **iframe 임베드 가능 여부는 5240 측 책임.** 5240 응답에 `X-Frame-Options: DENY` 나 강한 `CSP frame-ancestors` 가 걸려 있으면 iframe 으로 못 띄움 → `window.open` 팝업 폴백 필요. 5240 측에 서비스데스크 도메인의 **frame-ancestors 허용** 요청은 사전 합의 필수.
- **딥링크 URL 은 서버가 동적으로 내려준다.** 위젯에 URL 하드코딩 시 5240 도메인 변경마다 위젯 재배포가 필요. 메시지 응답의 `metadata.buttons[].url` 형태로 서버가 내려주고 위젯은 그대로 따라가는 구조 권장. (3.5 의 `BUTTON_MAP` 은 fallback / 정적 매핑 용도로만 활용)
- **결과 피드백 경로 부재.** 사용자가 5240 화면에서 작업을 완료해도 위젯/서비스데스크 서버는 그 결과를 직접 알 수 없다. 5240 배치 → RPA API(`HR_APPROVAL_COMPLETED` 등) → 위젯 푸시 경로로 한 번 더 돌아 들어와야 채팅창에 결과가 반영됩니다. 이 비동기 지연을 UX 에 반영(예: "처리 중입니다" 안내) 필요.
- **팝업 차단 대응.** iframe 폴백으로 `window.open` 을 쓸 때는 반드시 사용자 클릭 핸들러 안에서 직접 호출해야 차단 회피가 가능합니다. 비동기 작업(서버 응답 대기) 이후 `window.open` 을 부르면 차단됩니다.

4. 서버 ↔ 위젯 통신 구조

4.1 통신 채널 선택: WebSocket 권장

방식	장점	단점	결론
WebSocket	양방향, 저지연, 표준	인프라(LB sticky/Redis pub-sub) 필요	✅ 권장
SSE (EventSource)	구현 간단, HTTP/2 친화	서버→클라 단방향 (클라→서버는 별도 REST)	△
Long Polling	방화벽 친화	비용 높고 지연 큼	❌

4.2 연결 라이프사이클



4.3 메시지 프로토콜 (예시)

모든 WebSocket 메시지는 JSON `{ type, payload }` 형태로 통일합니다.

서버 → 위젯

```
{ "type": "message", "payload": {
  "messageId": "msg_001",
  "rpaCode": "HR_PC_ON",
  "content": "출근 처리하시겠습니까?",
  "metadata": {
    "buttons": [
      { "label": "출근하기", "action": "deeplink",
        "url": "https://api.5240.cloud/MobileMain.do" }
    ]
  },
},
"createDt": 1700000000000
}}
```

위젯 → 서버

```
{ "type": "user.message", "payload": { "text": "휴가" } }
{ "type": "suggestion.select", "payload": { "keyword": "휴가" } }
{ "type": "button.click", "payload": { "messageId": "msg_001", "buttonIdx": 0 } }
{ "type": "history.fetch", "payload": { "lastSortKey": 1699999999000 } }
```

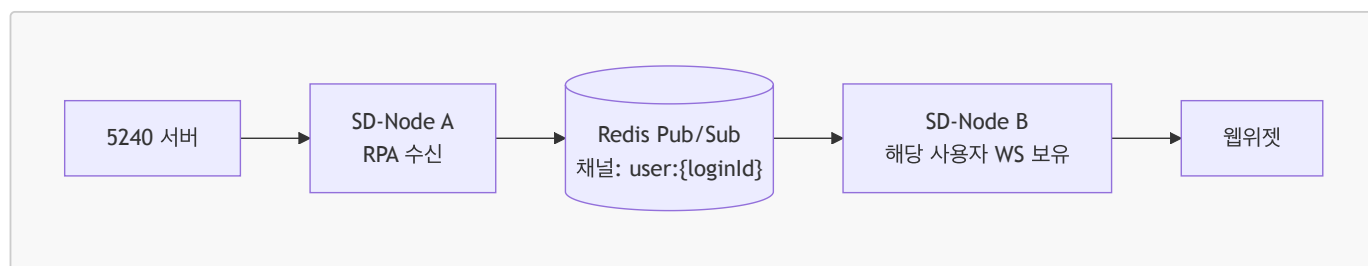
4.4 미수신 메시지 처리 (오프라인 보강)

웹위젯이 연결되어 있지 않은 동안 도착한 RPA 메시지는 DB에 쌓여 있어야 하며, 재접속 시 **history.fetch** 로 모두 수신할 수 있어야 합니다.

→ **sort_key**(또는 **messageId**) 기반으로 "마지막으로 받은 것 이후"만 조회하는 API를 두면 됩니다.

4.5 멀티노드 푸시 구조

서비스데스크 서버를 2대 이상 운영하면, RPA가 도착한 노드 ≠ 사용자가 WebSocket으로 붙은 노드인 경우가 생깁니다. → **Redis Pub/Sub** 으로 노드 간 fan-out:



5. 인증·보안 (반드시 챙길 것)

항목	권장
RPA API 인증	5240 ↔ SD 간 공유 시크릿(API Key 헤더) + IP allowlist. 양쪽 합의 필요.
위젯 WebSocket 인증	짧은 TTL(예: 5분) JWT. 만료 시 위젯이 갱신.
CORS	위젯 origin과 SD 서버 origin이 다르면 CORS 명시 허용.
CSP / X-Frame-Options	5240 화면을 iframe으로 띄우려면 5240 측에 frame-ancestors 허용 요청.
XSS 방지	채팅 메시지 렌더링 시 HTML 이스케이프. dangerouslySetInnerHTML 금지.

항목	권장
민감정보 로깅 금지	<code>loginId</code> , 메시지 본문 등을 평문으로 외부 로그에 남기지 말 것.

6. 에러 처리 / 운영 시나리오

6.1 RPA API 측

케이스	응답	후속
잘못된 <code>rpaCode</code>	<code>code: 4001</code> , message: "INVALID_RPA_CODE"	메시지 미저장
알 수 없는 <code>loginId</code>	<code>code: 4004</code> , message: "USER_NOT_FOUND"	메시지 미저장, 알람 기록
중복 호출 (idempotency)	<code>code: 0</code> , 기존 <code>messageId</code> 반환	사이드이펙트 없음
내부 오류	<code>code: 5000</code>	5240 재시도 정책에 의존 → 반드시 재시도 안 전성 확보

6.2 웹위젯 측

- WebSocket 끊김 → 지수 backoff (1s, 2s, 4s, 최대 30s) 로 재연결
- 재연결 성공 시 `history.fetch` 호출로 누락 메시지 복원
- 딥링크 호출 후 5240 화면이 401(로그인 만료) 반환 → 사용자에게 재로그인 안내

7. 테스트 전략

레벨	도구 후보	무엇을 테스트하나
단위	Jest / JUnit	<code>rpaCode</code> → <code>ButtonCard</code> 변환 로직, intent 매칭
통합	Testcontainers (MySQL/Redis)	RPA POST → DB 저장 → Redis publish 까지
E2E (서버)	Postman/Newman, Pact	5240 ↔ SD RPA API 계약 검증
E2E (위젯)	Playwright	5240 모의 페이지에 위젯 임베드 → 채팅/버튼 클릭 시나리오
부하	k6 / Artillery	WebSocket 1만 동시 연결, RPA POST burst

✅ 꼭 작성할 테스트 케이스

1. 같은 (`loginId`, `rpaCode`) 가 5초 내 2번 도착해도 중복 메시지가 안 쌓이는가
2. 위젯이 오프라인 상태에서 도착한 메시지를 재접속 시 누락 없이 받는가
3. WebSocket 비정상 종료 시 60초 내 자동 재연결 되는가
4. 멀티노드 환경에서 다른 노드의 사용자에게도 푸시가 전달되는가

8. 개발 진행 순서 (체크리스트)

주니어 개발자는 아래 순서로 진행하면 길을 잃지 않습니다.

- ☐ 0 — 5240 측 담당자와 RPA API 인증/Content-Type/재시도 정책 합의, 샘플 페이로드 확보
- ☐ 1 — 서버: RPA API 스케레톤 + DB 스키마 + Postman 으로 200 OK 확인

- ☐ 2 — 위젯: 5240 mock 페이지에 빈 채팅 UI 임베드, loader.js 작동 확인
 - ☐ 3 — 서버: 메시지 저장 + intent 변환(rpaCode → buttonCard) + REST 이력 조회
 - ☐ 4 — 위젯: WebSocket 연결 + 메시지 수신 → 버튼 카드 렌더
 - ☐ 5 — 위젯: 추천단어 자동완성 + 버튼 클릭 → 딥링크 iframe
 - ☐ 6 — 서버: 멀티노드 push (Redis pub/sub)
 - ☐ 7 — 인증/CSP/CORS/보안 점검 + E2E 테스트 작성
 - ☐ 8 — 5240 staging 환경과 연동 시험 + 부하 테스트
-

9. 자주 하는 실수 (FAQ)

1. "5240 화면을 iframe 으로 띄웠는데 안 보임" — **X-Frame-Options: DENY** 가 원인. 5240 측에 **frame-ancestors** 허용 요청 필요.
 2. "rpaCode 매핑을 위젯 코드에 하드코딩" — 운영 중 추가/수정이 잦으면 서버에서 metadata로 내려주는 편이 안전합니다.
-

10. 참고 문서

- [prompts/01.archetecture.md](#) — 원본 아키텍처/요구사항
 - [자체챗봇_연동_개발자_PRD.md](#) — 유사 시스템(KiwiBox) 챗봇 연동 PRD (인터페이스 계약 참고용)
 - [Kori_챗봇_연동_PRD.md](#) — 외부 챗봇 Kori 연동 PRD (구현 패턴 벤치마크)
 - [2.TIGRIS_CHAT BOT_API_v1.2_사용중인 api 추출.xlsx](#) — 실제 운영 중 API 목록
-